

A. What is an AI agent, really?

Starting from nothing. No prior knowledge assumed.

An **AI agent** is a small program with a job description written in plain English, powered by an AI model. You do not program it with rules. You tell it what its job is, what tools it can use, and let it work out the rest.

In real life — a new employee on their first day

You do not hand a new hire ten thousand lines of instructions. You say: *“You handle customer refunds. Here is the refund system, here is the order database. Ask Priya in finance if the amount is over £500.”*

That is an agent. A job description, some tools, and permission to ask a colleague. The AI model is the part that reads the situation and decides what to do next.

And an agent network?

One agent doing everything is like one person running a whole restaurant — possible, slow, and they will drop something. So you use several, each with a narrower job, and one of them takes the order and decides who to pass it to.

In real life — a restaurant kitchen

A customer asks for a dish. The **head chef** does not cook it alone — they pass the fish to the fish station, the sauce to the sauce cook, the dessert to pastry, then plate it up and send it out.

The customer only ever speaks to one person. Behind that, four specialists did the work. **That is an agent network**, and the head chef is what this project calls the “front man”.

B. The problem this project solves

One sentence, and it is the whole reason the project exists.

Cognizant AI Lab publishes a free framework called **neuro-san** that builds these agent networks for you. Describe the job in one sentence and it designs a whole team of agents in about five seconds. It is genuinely impressive.

It designs one team. It never checks whether the team is any good.

There is **no scoring function anywhere in the framework**. So nobody can answer the obvious questions: is a nine-agent team better than a five-agent one for this job? Should every agent use the expensive AI model, or would a cheap one do for most of them? Are the job descriptions it wrote actually clear?

It is design with no measurement.

In real life — an architect who never checks the building stands up

Imagine an architect who can sketch a house in five seconds. Brilliant. Now imagine they never calculate whether the roof holds, never price the materials, and never compare their design to any other.

You would not call that finished. You would say the *drawing* is done and the *engineering* has not started. That is exactly the gap here — and this project is the engineering half.

Why it matters in money

Two agent teams can give you **the same answers** and send you very different bills. AI providers charge by the amount of text processed, so a badly-shaped team quietly costs more forever, and nothing tells you.

In real life — two electricians, same job

Both rewire your house. Both pass inspection. One takes four hours, the other takes seven and charges you for it.

Identical result, very different invoice. You would want to know that *before* hiring, not after. Right now, with AI agent teams, there is no way to know.

C. The clever bit, in one picture

Why the method is called ESP, and why it is affordable.

Testing an agent team for real is slow and costs money — about **eight minutes** and a real bill each time. Trying thousands of ideas that way is impossible.

In real life — designing a car in a wind tunnel

Real wind-tunnel tests are slow and expensive, so car engineers do this:

- 1. Test a handful of real shapes in the tunnel. Slow, costly.
- 2. Use those results to build a cheap computer simulator that *guesses* how a shape will perform.
- 3. Try ten thousand shapes in the simulator overnight. Free, instant.
- 4. Put only the best few back in the real tunnel.
- 5. Feed those new real results back into the simulator, so its guesses get better. Repeat.

That is precisely what this project does, with agent teams instead of car bodies. The method has a name — **ESP**, Evolutionary Surrogate-assisted Prescription — and it was invented by Cognizant AI Lab, the same people who wrote neuro-san. The “surrogate” is the simulator.

The numbers behind the picture, measured here

	Real test (the wind tunnel)	The simulator's guess
One team	about 8 minutes, real money	a fraction of a second
2,000 teams	roughly 267 hours	about 0.14 seconds
Cost	a real bill each time	nothing at all

This is the whole trick

Trying two thousand designs for real would take months and a large bill. Trying two thousand *guesses* takes a tenth of a second and costs nothing. You then pay for only the handful the guess says are worth it.

The guessing is free. That is what makes searching affordable at all.

D. How it works, step by step

Six steps. No jargon.

1. Invent a company that does not exist

A fictional logistics firm: 24 depots, 40 contracts, 60 incident reports, 124 documents. All made up, on purpose. If we used a real company, the AI might already know the answers from its training and would look clever without actually reading anything. Nothing about this company exists anywhere, so the only way to answer is to go and look it up.

In real life — an exam with a made-up textbook

If you set an exam on famous history, a student can bluff from general knowledge. If you set it on a novel written last week that nobody has read, the only way to pass is to actually read the book. That is why the company is invented.

2. Write 17 questions where you already know the answer

They range from easy — “*who manages depot D08?*” — to hard, needing four documents joined together: “*incident INC-4413 hit a contract, that contract is served by a depot, who manages that depot?*” Because the company and the questions come from the same generator, every answer is correct by construction. Nobody hand-wrote an answer that might be wrong.

3. Score a team by actually running it

Give the team all 17 questions and count three things: how many it got right, how much text it burned through (the bill), and how many agents it used. Getting the answer right matters most — a cheap team that answers nothing is worthless — but cost and size count too.

4. Change one thing and see what happens

Seven kinds of change: add an agent, remove one, change who reports to whom, split one agent into two, merge two into one, change who is allowed to search the documents, and swap an agent onto a cheaper or more expensive AI model. Each change is checked for sanity first, and a broken one is **thrown away rather than patched up**.

In real life — reorganising a team at work

Hire someone. Let someone go. Change who reports to whom. Split one overloaded role into two. Merge two quiet ones. Give someone access to the shared drive. Move someone from a contractor rate to a staff rate.

Those are exactly the seven changes — and just like at work, most of them make things slightly worse and a few make things much better. You only find out by trying.

5. Learn to guess the score without running anything

Look at the *shape* of a team — how many agents, how deep the chain of command, who talks to whom, which model each one uses — and predict how well it will score. This is the simulator from the wind tunnel. It learns from the real results collected so far, so it starts out poor and improves as more real results arrive.

6. Try thousands, pay for a few, repeat forever

Generate thousands of changed teams, guess all their scores for free, keep the best few, and pay to really run only those. Their real results go back into training the guesser, which then guesses better next time.

E. What it actually found

The finding that makes the whole thing worth doing.

Twelve different team designs were run for real against all 17 questions — three written by hand, nine invented by the search. The important column is the last one.

The design	Questions right	Cost (text processed)	Agents
One agent doing everything	14 of 17	expensive	1
A flat pair of agents	14 of 17	middle	3
The shape neuro-san's designer produces	14 of 17	the dearest	4
The best one the search found	16 of 17	cheaper than the dearest	5

It answered two more questions, and cost less doing it.

The design the search found beat all three hand-written ones on both counts at once: two more questions right, and **7% less** text processed than the shape neuro-san's own designer produces.

What changed was not a new agent or a new tool. It was **which AI model one agent inside the team uses** — a dial the framework already has, which nobody turns, because until now nothing could tell you what turning it was worth.

The three hand-written designs are the same lesson in miniature: they got *exactly* the same number right as each other for different money, and across all twelve designs the dearest cost **95% more** than the cheapest. On a real system answering thousands of questions a day, that is a bill nobody can see, because the framework cannot measure any of this.

In real life — your electricity bill

Two houses, same size, same family, same comfort. One pays double. Not because anyone is doing anything differently — because of insulation and boiler choices nobody ever measured.

Nobody thinks about it until somebody finally measures it. This project is the meter.

A warning about precise numbers

Earlier versions of this document quoted exact figures and an exact percentage. Those specific numbers turned out to be partly an **artifact of a measurement bug** — questions that timed out were being counted as wrong answers, which made three different outcomes look identical. After the fix, the scores moved.

What survived the fix is the part that matters: **the cost ordering is stable, and the gap is large**. The cheapest design really is far cheaper. But quote the ordering, not a decimal.

F. Why it runs by itself, forever

The part that makes it a product rather than a school project.

The AI provider gives away a limited amount free: about **500 requests a day**. Testing one team costs about 165 of them. So a whole day buys **three tests**.

The first version of this project was a script that planned forty tests and ran them all at once. It failed every single time — not from a bug, but from arithmetic. Forty tests need about thirteen days of allowance.

The change of mind that fixed it

The daily limit is not an obstacle to a service. It is its **rhythm**.

Instead of one big run that dies, it now wakes up every hour, spends whatever allowance is left, writes down what it learned, and stops. Tomorrow it carries on. Over weeks it builds up a collection of results that could never be bought in one afternoon — and nobody has to be watching.

In real life — saving up rather than borrowing

You cannot buy the thing this month. You can put aside what is spare each month and have it by spring. Nothing clever — but it needs a system that survives you forgetting about it, and a record that survives being interrupted halfway.

That is why it saves after *every single test* rather than at the end. Being interrupted is the expected ending, not a failure.

G. You can talk to it

The winner is a thing you can use, not a number in a table.

A search that finishes with a winning identifier in a spreadsheet has stopped one step short.

The reason to measure teams is that one of them is better to **actually use**.

One command puts the best-measured team behind a web page. You type a question, the real agents run, and if your question happens to be one of the 17 graded ones, it is **marked against the known answer in front of you** — right or wrong, no hiding.

Talking to a measured agent network

`designer_shaped` — the shape neuro-san's own designer produces. No measurements are loaded in this deployment, so this is the baseline rather than a measured winner.

Ask

Try one of these — each has a known correct answer:

Who is the depot manager of Meridian Logistics depot D08?

Which city is the depot that services contract C-2100 located in?

Incident INC-4413 affected a contract serviced by a depot. Who is that depot's manager?

```

answer: J. Vasquez
agents: 4
seconds: 2.8
expected: J. Vasquez -> CORRECT

```

What this is. Meridian Logistics is invented — 24 depots, 40 contracts, 124 documents. Nothing about it is in any model's training data, so the only way to answer is to go and read, which is what makes the score a measurement of the topology rather than of recall.

What it is not. No evolved candidate has beaten this baseline yet, and the surrogate has not trained — it needs 8 measurements and there are fewer. Both are stated in the repository rather than left out.

Source: github.com/Sivakumarraj/neuro-san-esp

The page, answering. Four agents, and the answer graded correct against the known ground truth.

A four-hop question, answered correctly, live

Incident INC-4413 affected a contract serviced by a depot. Who is that depot's manager?

The team had to find the incident, follow it to a contract, follow that to a depot, then look up who runs it — four documents, chained. It answered **R. Delacroix**, which is correct, in about 48 seconds.

H. What it has not done

The part most documents leave out.

It found a better team once, on a very small experiment

Twelve real tests exist and the search is only a couple of rounds deep. A few rounds, one throw of the dice, one kind of question, one AI model — not repeated, and not tried on questions it had never seen before. Being *sure* a design is better needs a collection of results, a collection needs allowance, and allowance arrives at three tests a day. That is the honest state of it. Had it found nothing better, that would be reported as the answer.

The guesser helps on paper; the real test has not been run

It needs eight real results before it can learn anything at all. At the round that produced the winning design it had nine, and when tested it guessed the order of designs **worse than picking at random**. So that round's improvement came from the searching, not the guessing. Given every result collected since, it does better than random: shown three designs it has never seen, it picks the best one about six times in ten, where picking blindly manages three. So does the simple rule of picking the biggest team, which learns nothing, so the guesser has not yet shown it knows more than that. The latest design is the first one it actually picked. What has not been run is the whole search with the guesser and without it on the same budget, which is what would settle whether the guesser is worth its place.

Below eight results it returns the same guess for every design, and the code now says so instead of printing a confident-looking number.

An earlier version printed `+0.000` in that case, which looked exactly like a measured result of zero. It was a placeholder. That was quoted as a real measurement for a while — including by me — and it was wrong.

Things learned only by running it

Roughly a dozen bugs were found this way, and nearly all of them made a **good** design look bad. That is the dangerous kind, because a search being fed wrong information still produces a smooth, convincing-looking graph.

The wrong agent was answering the question

The framework treats whichever agent is listed *first* as the one who receives the question. Listing them alphabetically meant one design was fronted by an arithmetic agent, which replied “you did not provide any numbers” to all 17 questions and scored zero. It had never actually been run as designed — and neither had any other measurement taken before that.

Timeouts were being counted as wrong answers

A question that ran out of time looked identical to a question answered incorrectly. That made three quite different designs appear to score exactly the same, which became a headline finding. It was an artifact.

A test certified a bug instead of catching it

A function was supposed to use world time and used local time instead. The test that was meant to check this asserted local time too — so it passed, permanently, while being wrong.

A missing ingredient nobody could notice locally

A required package was never declared. It worked forever on the machine that happened to have it already. Only a completely fresh install could reveal it.

In real life — the smoke alarm you never tested

It looks fine. It has a light on it. You would only discover it does nothing on the one night it matters.

Most of these bugs were that: something that looked like it was working, reporting confident numbers, and quietly wrong. Which is the argument for measuring in the first place.

I. Every word, in plain English

In case any of them were new.

Word	What it means
AI model	The engine that reads text and writes text. You rent it by the word.
Agent	A small program with a plain-English job description, powered by a model. Like a new hire with a role and some tools.
Agent network	Several agents wired together, one of whom takes the question and asks the others. Like a restaurant kitchen.
Topology	The <i>shape</i> of that team: how many, and who talks to whom.
neuro-san	Cognizant AI Lab's free framework for building agent networks. This project sits on top of it.
ESP	Evolutionary Surrogate-assisted Prescription. Cognizant AI Lab's method: build a cheap guesser, search against it for free, pay only for the best few. The wind tunnel.
Surrogate / guesser	The cheap stand-in that estimates a score without running anything.
Fitness	The score a design gets: right answers, cost, and team size combined.
Token	Roughly a word. Providers bill by these, so “tokens” and “cost” mean the same thing here.
Quota / rate limit	The provider's cap on free use. Here: about 500 requests a day.
Baseline	The design you are trying to beat — the one neuro-san's own designer would have handed you.

The whole project in one sentence

neuro-san can design a team of AI agents but cannot tell you whether the team is any good; this measures that, searches for a better team, keeps searching by itself, and lets you talk to the winner — and the first thing it measured was that two teams doing the same job can cost twice as much as each other.

Code and full technical write-up: github.com/Sivakumarraj/neuro-san-esp